Pokhara University **Faculty of Science and Technology**	
Course Code: CMP 519 (3 Credit)	Full Marks: 100
Course Title: Software Engineering	Pass Mark: 60
Nature of the Course: Theory	Total Lecture Hours: 45 hours
Level: Master	Program: Computer Science/ Computer Engineering

1. Course Description:

The main objective of this course is to provide required knowledge on the various issues of software project management and related tasks including planning, design, development, implementation, maintenance and cross life cycle activities using object oriented concepts and models.

2. General Objectives:

- Understand and apply the fundamental principles of software engineering and Unified Modeling Language (UML) basics, in the development of software solutions.
- Analyze and specify software requirements, develop use cases and scenarios, apply in object oriented software engineering
- Analyze and Evaluate Software Maintenance and Evolution Strategies
- Apply Advanced Object-Oriented Software Engineering

3. Contents in Detail

Specific Objectives	**Course Contents**
	Unit I: Introduction (5 hrs.)
● Understanding the Software engineering and requirement engineering	1.1 Software Engineering Concepts ● Participants and Roles in Software Projects ● Systems and Models ● Work Products ● Activities, Tasks, and Resources ● Notations, Methods, and Methodologies 1.2 Overview of Software Process Models ● Traditional Models: Waterfall, Iterative, Spiral ● Agile Models

	● Comparison of Process Models 1.3 Introduction to Requirements Engineering ● Functional Requirements ● Non-functional Requirements ● Importance of Requirements Engineering
	Unit II: Software Process Models and Project Management (6 Hours)
● Understanding the principles of project management and process management	2.1 Agile Development Principles ● Manifesto for Agile Software Development ● Benefits of Agile Methodologies 2.2 Scrum and Kanban Methodologies ● Scrum Roles, Artifacts, and Ceremonies ● Kanban Boards and Workflows 2.3 DevOps Culture and Principles ● Continuous Integration/Continuous Deployment (CI/CD) ● Collaboration between Development and Operations 2.4 Project Organization Concepts ● Project Organizations and Roles ● Task Assignment and Work Products ● Scheduling and Time Management 2.5 Project Communication ● Planned vs. Unplanned Communication ● Communication Mechanisms and Tools ● Meetings and Reviews 2.6 Introduction to DevOps Tools ● Overview of Jenkins, Docker, and Kubernetes
	Unit III: Requirements Engineering and Elicitation (6 Hours)

● Understanding the principles of requirement elicitation and requirement engineering	3.1 Requirements Elicitation Concepts ● Importance of Stakeholder Involvement ● Challenges in Requirements Gathering 3.2 Detailed Requirements Analysis ● Completeness and Consistency ● Clarity and Correctness ● Realism and Verifiability ● Traceability of Requirements 3.3 Elicitation Activities ● Identifying Actors and Stakeholders ● Developing Scenarios and Use Cases ● Refining Use Cases and User Stories ● Identifying Relationships among Actors and Use Cases 3.4 Managing Requirements ● Maintaining Traceability ● Negotiating Specifications with Clients ● Documenting Requirements
	Unit IV: Object-Oriented Analysis and Design with UML (8 Hours)
● Understanding the principles of object oriented analysis and design activities through UML	4.1 OOAD Principles ● Encapsulation ● Abstraction ● Inheritance ● Polymorphism 4.2 Analysis Object Models and Dynamic Models ● Entity, Boundary, and Control Objects ● Generalization and Specialization 4.3 UML Diagrams ● Use Case Diagrams ● Class Diagrams

	● Sequence Diagrams ● Activity Diagrams ● State Machine Diagrams 4.4 Modeling Interactions and Object Lifecycles ● Mapping Use Cases to Objects ● Modeling Associations and Aggregations ● Modeling Inheritance Relationships 4.5 Reviewing Analysis and Design Models ● Iterative Refinement ● Client Sign-Off Procedures 4.6 Case Study ● Applying OOAD and UML to a Real-World Application
	Unit V: Design Patterns and Reuse (5 Hours)
● Understanding the principles of software design patterns and reuse activities.	5.1 Reuse Concepts ● Solution Objects and Reusable Components ● Specification vs. Implementation Inheritance ● Delegation and the Liskov Substitution Principle 5.2 Common Design Patterns ● Creational Patterns: Factory, Abstract Factory ● Structural Patterns: Adapter, Composite, Bridge ● Behavioral Patterns: Strategy, Command ● 5.3 Selecting and Applying Design Patterns ● Heuristics for Pattern Selection ● Integrating Patterns into Designs 5.4 Managing Reuse ● Documenting Reuse Strategies

	● Assigning Responsibilities for Reusable Components
	Unit VI: Software Quality Assurance and Testing (5 Hours)
● Understanding the principles of Software Quality Assurance and Testing	6.1 Software Quality Concepts and Attributes ● Reliability, Usability, Efficiency, Maintainability ● Software Safety and Security 6.2 Software Cost Estimation ● Introduction to the COCOMO Model ● Estimation Techniques and Tools 6.3 Software Quality Assurance Planning and Process ● Quality Assurance Activities ● Role of QA in the Software Lifecycle 6.4 Testing Concepts ● Faults, Erroneous States, and Failures ● Test Cases, Test Stubs, and Drivers 6.5 Testing Activities ● Unit Testing ● Integration Testing ● System Testing ● Regression Testing 6.6 Automated Testing Frameworks ● Introduction to Tools like Selenium and JUnit ● Integrating Testing Tools with CI/CD Pipelines
	Unit VII: Configuration Management and DevOps Tools (4 Hours)
● Understanding the Configuration Management and DevOps Tools	7.1 Configuration Management Planning ● Importance of Configuration Management ● Configuration Items and Baselines 7.2 Change Management

	● Managing Change Requests ● Version and Release Management 7.3 System Building ● Build Automation ● Continuous Integration Practices 7.4 CASE Tools for Configuration Management ● Overview of Tools like Git and Subversion 7.5 DevOps Tools in Configuration Management ● Advanced Use of Jenkins, Docker, and Kubernetes
	Unit X: Emerging Trends in Software Engineering (8 Hours)
● Understanding the role of emerging technologies in Software Engineering	8.1.1 Cloud-Native Development ● Microservices Architecture ● Principles and Benefits ● Designing Microservices ● Serverless Computing ● Concepts and Use Cases ● Advantages and Limitations 8.1.2 AI in Software Engineering ● AI-Assisted Coding and Testing ● Tools and Technologies ● Impact on Development Processes ● Machine Learning Applications ● Incorporating ML into Software Solutions 8.1.3 Software for IoT and Edge Computing ● Challenges and Best Practices ● Designing for Resource-Constrained Environments 8.1.4 Sustainability in Software Engineering ● Eco-Friendly Practices

	● Energy-Efficient Coding ● Sustainable Resource Usage ● Green Coding ● Techniques and Benefits 8.1.5 Secure Software Development Practices ● Basic Principles of Writing Secure Code ● Common Vulnerabilities and Mitigation Strategies ● Understanding Common Web Security Risks ● Brief Strategies for Prevention ● Integrating Security Testing into Development

4. Methods of Instructions:
1.Lectures and conceptual explanations 2. Demonstrate the application of research techniques in real-world applications 3. Assist students to gain practical skills in data collection, analysis, and software tools 4. Individual research assignments 5. Classroom discussions

6: Case Study:

An individual case study should be given to each student on a software project and should be analyzed with any UML CASE tool (like Rational Rose Enterprise Suit 2000) and implemented in OO. 25 % to 50% of sessional marks should be allocated for evaluation.

7. Evaluation System and Students' Responsibilities

Evaluation System
The internal evaluation of a student may consist of assignments, attendance, term-exams, lab reports and projects etc. The tabular presentation of the internal evaluation is as follows:

Internal Evaluation	**Weight**	**Marks**	**External Evaluation**	**Marks**
Attendance & Class Participation	10%	60	Semester-End examination	40
Assignments	20%			

Internal Assessment	40%			
Research/Project/Case Study Work	30%			
Total Internal		60	**Total External**	40
Full Marks: 60 + 40 = 100				

Students' Responsibilities
Each student must secure at least 60% marks separately in internal assessment and practical evaluation with 80% attendance in the class in order to appear in the Semester End Examination. Failing to get such score will be given NOT QUALIFIED (NQ) to appear the Semester-End Examinations. Students are advised to attend all the classes, formal exam, test, etc. and complete all the assignments within the specified time period. Students are required to complete all the requirements defined for the completion of the course.

8. Prescribed Books and References

Text Books:
1. Software Engineering: A Practitioner's Approach, "Roger S. Pressman
2. Bernd Bruegge & Allen H. Dutoit, **Object-Oriented Software Engineering Using UML, Patterns, and Java™**

References:
1. Software Engineering, " Ian Sommerville", ninth edition
2. Software Engineering Fundamentals, " Ali Behforooz and Frederick J. Hudson